ADC: INTR Register

Offset: 0x14

Description

Raw Interrupts

Table 1126. INTR Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	**FIFO**: Triggered when the sample FIFO reaches a certain level. This level can be programmed via the FCS_THRESH field.	RO	0x0

ADC: INTE Register

Offset: 0x18

Description

Interrupt Enable

Table 1127. INTE Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	**FIFO**: Triggered when the sample FIFO reaches a certain level. This level can be programmed via the FCS_THRESH field.	RW	0x0

ADC: INTF Register

Offset: 0x1c

Description

Interrupt Force

Table 1128. INTF Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	**FIFO**: Triggered when the sample FIFO reaches a certain level. This level can be programmed via the FCS_THRESH field.	RW	0x0

ADC: INTS Register

Offset: 0x20

Description

Interrupt status after masking & forcing

Table 1129. INTS Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	**FIFO**: Triggered when the sample FIFO reaches a certain level. This level can be programmed via the FCS_THRESH field.	RO	0x0

12.5. PWM

12.5.1. Overview

Pulse width modulation (PWM) smoothly varies the average voltage of a digital signal using controlled-width positive pulses at regular intervals. The fraction of time spent high is known as the duty cycle. This may be used to approximate an analogue output or control switchmode power electronics.

The RP2350 PWM block has 12 identical slices. Each slice can drive two PWM output signals, or measure the frequency or duty cycle of an input signal. The two outputs on each slice have the same period, but independently varying duty cycles, so this gives a total of 24 controllable PWM outputs in the QFN-80 package.

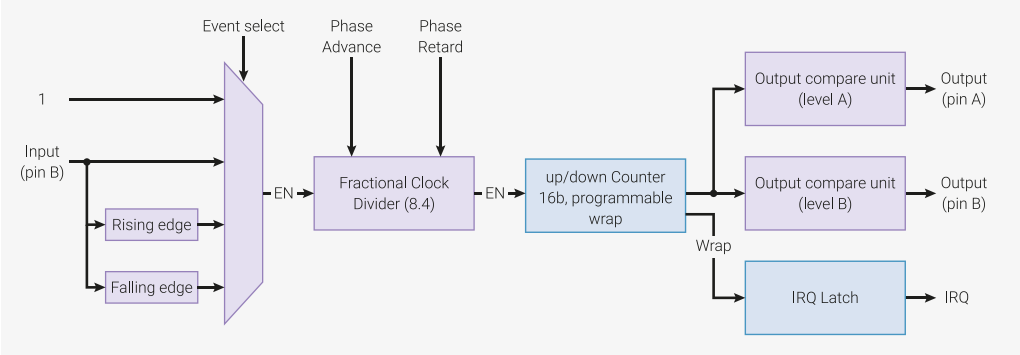

Figure 111. A single PWM slice. A 16-bit counter counts from 0 up to some programmed value, and then wraps to zero, or counts back down again, depending on PWM mode. The A and B outputs transition high and low based on the current count value and the preprogrammed A and B thresholds. The counter advances based on a number of events: it may be free-running, or gated by level or edge of an input signal on the B pin. A fractional divider slows the overall count rate for finer control of output frequency.

Each PWM slice is equipped with the following:

- 16-bit counter
- 8.4 fractional clock divider
- Two independent output channels, duty cycle from 0% to 100% **inclusive**
- Dual slope and trailing edge modulation
- Edge-sensitive input mode for frequency measurement
- Level-sensitive input mode for duty cycle measurement
- Configurable counter wrap value
  - Wrap and level registers are double buffered and can be changed race-free while PWM is running
- Interrupt request and DMA request on counter wrap
- Phase can be precisely advanced or retarded while running (increments of one count)

Slices can be enabled or disabled simultaneously via a single global control register. Slices then run in lockstep, so that more complex power circuitry can be switched by the outputs of multiple slices.

12.5.1.1. Changes from RP2040

- Increased the number of slices from 8 to 12, with the 4 additional slices available on GPIOs 32 through 47 in the QFN-80 package.
- Added a second shared interrupt line (controlled by IRQ1_INTE), to aid use of PWM slices as simple repeating timers.

12.5.2. Programmer's model

All GPIO pins on RP2350 can be used for PWM:

Table 1130. Mapping of PWM channels to GPIO pins on RP2350. This is also shown in the main GPIO function table, Table 646

GPIO	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
PWM Channel	0A	0B	1A	1B	2A	2B	3A	3B	4A	4B	5A	5B	6A	6B	7A	7B
GPIO	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
PWM Channel	0A	0B	1A	1B	2A	2B	3A	3B	4A	4B	5A	5B	6A	6B	7A	7B
GPIO	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
PWM Channel	8A	8B	9A	9B	10A	10B	11A	11B	8A	8B	9A	9B	10A	10B	11A	11B

- The first 16 PWM channels (8 × 2-channel slices) appear on GPIOs 0 through 15, in the order `PWM0 A`, `PWM0 B`, `PWM1 A`, and so on.
- This pattern repeats for GPIOs 16 through 31. GPIO16 is `PWM0 A`, GPIO17 is `PWM0 B`, and so on up to `PWM7 B` on GPIO31. GPIO30 and above are available only in the QFN-80 package.
- The remaining 8 PWM channels (4 × 2-channel slices) appear on GPIOs 32 through 39, and then repeat on GPIOs 40 through 47.
- If you select the same PWM output on two GPIO pins, the same signal appears on both.
- If you use `B` pin as an input and select it on multiple GPIO pins, the PWM slice sees the logical OR of those two GPIO inputs.

NOTE

GPIOs 0 through 29 have the same channel assignment as RP2040 for pinout compatibility. This reduces the maximum number of independent PWM outputs in the QFN-60 package option of RP2350, but you can still use slices 8 through 11 for repeating timer interrupts in this package.

12.5.2.1. Pulse width modulation (PWM)

The PWM hardware continuously compares an input value to a free-running counter. This produces a toggling output; the amount of time spent at the high output level corresponds to the input value. The fraction of time spent at the high signal level is known as the duty cycle of the signal.

The counting period is controlled by the `TOP` register, with a maximum possible period of 65536 cycles, as the counter and `TOP` are 16 bits in size. Use the `CC` register to configure input values.

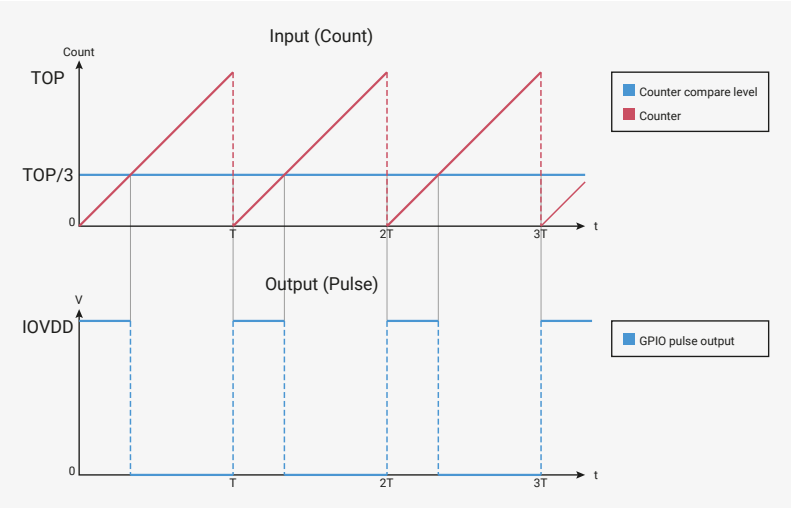

Figure 112. The counter repeatedly counts from 0 to TOP, forming a sawtooth shape. The counter is continuously compared with some input value. When the input value is higher than the counter, the output is driven high. Otherwise, the output is low. The output period T is defined by the TOP value of the counter, and how fast the counter is configured to count. The ***average*** *output voltage, as a fraction of the IO power supply, is the input value divided by the counter period (TOP + 1)*

This example shows the counting period and the A and B counter compare levels being configured on one of RP2350's PWM slices.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pwm/hello_pwm/hello_pwm.c Lines 14 - 29

```
14    // Tell GPIO 0 and 1 they are allocated to the PWM
15    gpio_set_function(0, GPIO_FUNC_PWM);
16    gpio_set_function(1, GPIO_FUNC_PWM);
17
18    // Find out which PWM slice is connected to GPIO 0 (it's slice 0)
19    uint slice_num = pwm_gpio_to_slice_num(0);
20
21    // Set period of 4 cycles (0 to 3 inclusive)
22    pwm_set_wrap(slice_num, 3);
23    // Set channel A output high for one cycle before dropping
24    pwm_set_chan_level(slice_num, PWM_CHAN_A, 1);
25    // Set initial B output high for three cycles before dropping
26    pwm_set_chan_level(slice_num, PWM_CHAN_B, 3);
27    // Set the PWM running
28    pwm_set_enabled(slice_num, true);
```

Figure 113 shows how the PWM hardware operates once it has been configured.

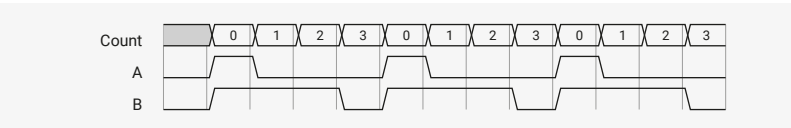

Figure 113. The slice counts repeatedly from 0 to 3, which is configured as the TOP value. The output waves therefore have a period of 4. Output A is high for 1 cycle in 4, so the average output voltage is 1/4 of the IO supply voltage. Output B is high for 3 cycles in every 4. The rising edges of A and B are always aligned.

By default, PWM slices count upward until they reach the value of the `TOP` register. After they reach the `TOP` value, they wrap to 0. Alternatively, set `CSR_PH_CORRECT` to 1 to enable **phase-correct mode**, where the counter counts downward after reaching `TOP`, until it reaches 0 again.

Phase-correct mode centres the pulse on the same point no matter the duty cycle; its phase is not a function of duty cycle. When phase-correct mode is enabled, the output frequency is halved. The slice spends two cycles at a count of `TOP` and two cycles at a count of 0 each PWM period.

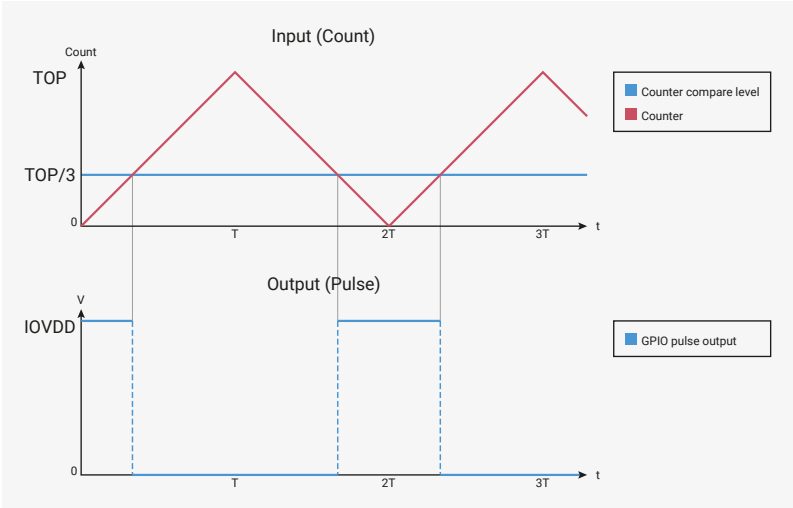

Figure 114. In phase-correct mode, the counter counts back down from TOP to 0 once it reaches TOP.

12.5.2.2. 0% and 100% Duty Cycle

The RP2350 PWM can produce toggle-free 0% and 100% duty cycle output.

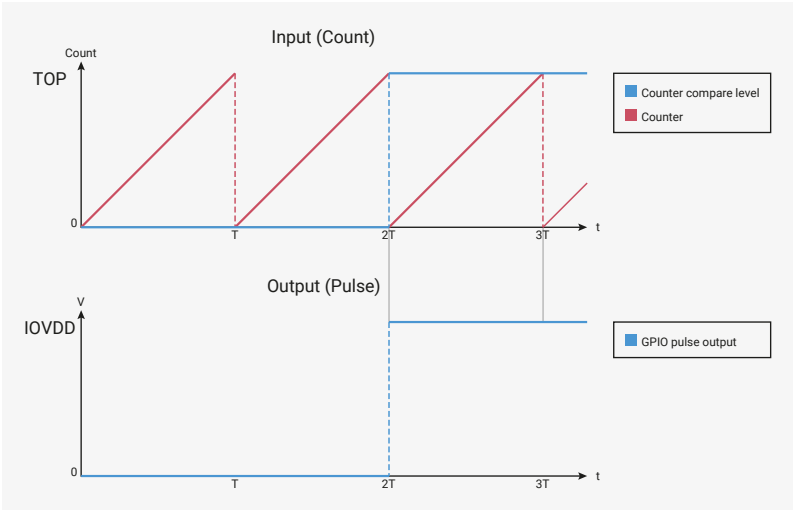

Figure 115. Glitch-free 0% duty cycle output for CC = 0, and glitch-free 100% duty cycle output for CC = TOP + 1

A `CC` value of 0 produces a 0% output: the output signal is always low. A `CC` value of `TOP` + 1 (equal to the period when not phase-corrected) produces a 100% output. If `TOP` is 254, the counter has a period of 255 cycles, and `CC` values in the range of 0 to 255 inclusive will produce duty cycles in the range 0% to 100% inclusive.

Glitch-free output at 0% and 100% helps avoid switching losses, for instance, when a MOSFET is controlled at its minimum and maximum current levels.

12.5.2.3. Double buffering

Figure 116 shows how a change in input value produces a change in output duty cycle. This can approximate analogue waveforms such as a sine wave.

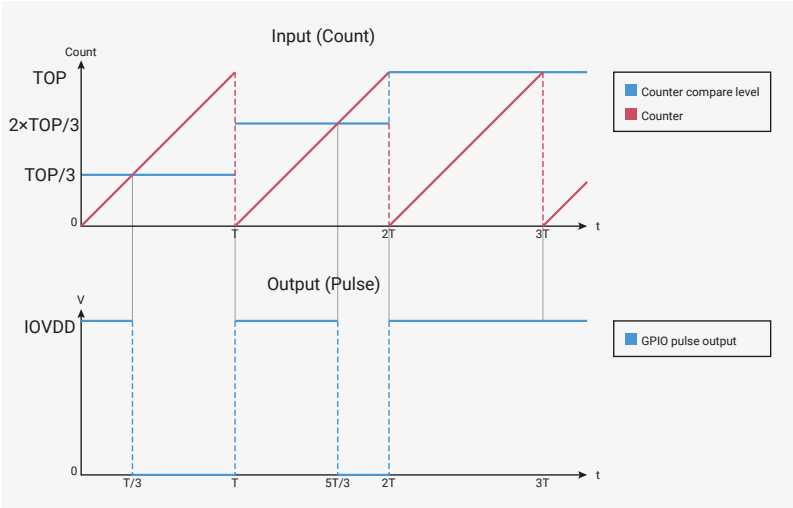

Figure 116. The input value varies with each counter period: first TOP / 3, then 2 × TOP / 3, and finally TOP + 1 for 100% duty cycle. Each increase in the input value causes a corresponding increase in the output duty cycle.

In Figure 116, the input value only changes at the instant where the counter wraps through 0. Figure 117 shows what happens if the input value is allowed to change at any other time: an unwanted glitch is produced at the output.

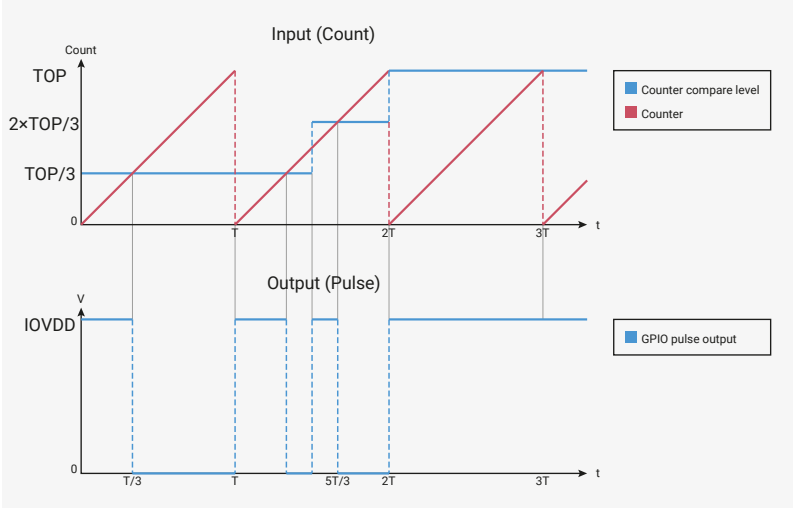

Figure 117. The input value changes whilst the counter is mid-ramp. This produces additional toggling at the output.

The behaviour becomes even more perplexing if the `TOP` register is also modified. It would be difficult for software to write to `CC` or `TOP` with the correct timing. To solve this, each slice has two copies of the `CC` and `TOP` registers: one copy that software can modify, and another, internal copy that is updated from the first register at the instant the counter wraps. Software can modify its copy of the register at will, but the changes are not captured by the PWM output until the next wrap.

Figure 118 shows the sequence of events where a software interrupt handler changes the value of `CC_A` each time the counter wraps.

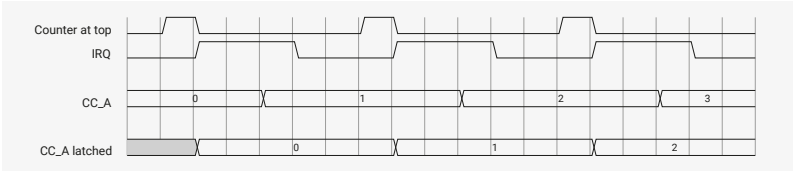

Figure 118. Each counter wrap causes the interrupt request signal to assert. The processor enters its interrupt handler, writes to its copy of the CC register, and clears the interrupt. When the counter wraps again, the latched version of the CC register is instantaneously updated with the most recent value written by software, and this value controls the duty cycle for the next period. The IRQ is reasserted so that software can write another fresh value to its copy of the CC register.

There is no limitation on what values can be written to `CC` or `TOP`, or when they are written. In normal PWM mode (`CSR_PH_CORRECT` is 0), the latched copies update when the counter wraps to 0, which occurs once every `TOP` + 1 cycles. In phase-correct mode (`CSR_PH_CORRECT` is 1), the latched copies update on the 0 to 0 count transition, when the counter stops counting downward and begins to count upward again.

12.5.2.4. Clock divider

Each slice has a 8 integer bit, 4 fractional bit fractional clock divider configured by the `DIV` register. The clock divider allows you to slow the count rate by a factor of up to 256. To do this, the PWM generates an enable signal that gates counter operation. This allows you to achieve output frequencies significantly lower than the system clock. For instance, from a 125MHz system clock, the clock divider can slow the count rate to approximately 7.5Hz. Lower frequencies than this require a system timer interrupt (Section 12.8).

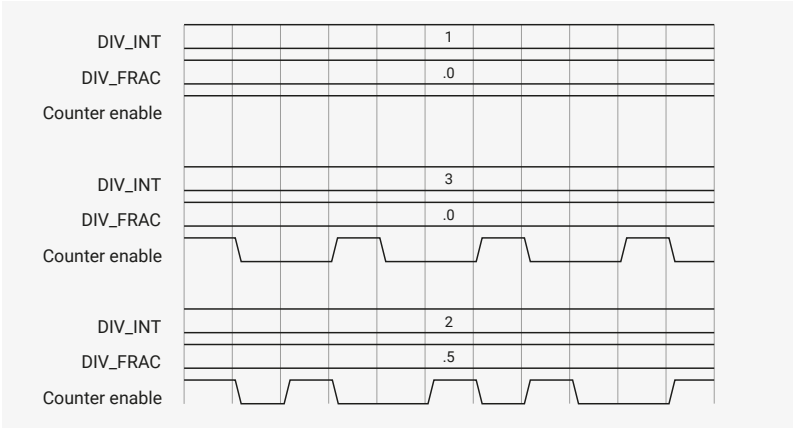

Figure 119. The clock divider generates an enable signal. The counter only counts on cycles where this signal is high. A clock divisor of 1 causes the enable to be asserted on every cycle, so the counter counts by one on every system clock cycle. Higher divisors cause the count enable to be asserted less frequently. Fractional division achieves an average fractional counting rate by spacing some enable pulses further apart than others.

The fractional divider is a first-order delta-sigma type.

The clock divider also extends the effective count range when using level-sensitive or edge-sensitive modes to take duty cycle or frequency measurements.

12.5.2.5. Level-sensitive and edge-sensitive triggering

The PWM provides the following counter modes:

- Default free-running, counting continuously whenever the slice is enabled (free-running)
- Count continuously when a high level is detected on the B pin (level sensitive)
- Count once with each rising edge detected on the B pin (rising edge-sensitive)
- Count once with each falling edge detected on the B pin (falling edge-sensitive)

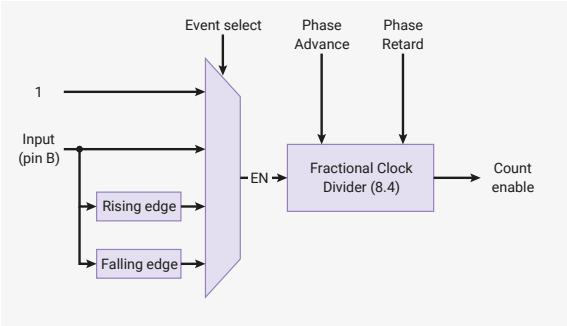

Figure 120. PWM slice event selection. The counter advances when its enable input is high. This enable is generated by two sequential stages. First, any one of four event types (always on, pin B high, pin B rise, pin B fall) can generate enable pulses for the fractional clock divider. The divider can reduce the rate of the enable pulses, before passing them on to the counter.

Use the `DIVMODE` field in each slice's `CSR` to select a mode. In free-running mode, the A and B pins are both outputs. In any other mode, the B pin becomes an input that controls counter operation. `CC_B` is ignored when not in free-running mode.

You can measure the duty cycle or frequency of an input signal by running the slice for a fixed amount of time in level-sensitive or edge-sensitive mode. Due to the type of edge-detect circuit used, the low period and high period of the measured signal must both be strictly greater than the system clock period when taking frequency measurements.

The clock divider still operates in level-sensitive and edge-sensitive modes. At maximum division (`DIV_INT` is 0), the counter only advances once per 256 high input cycles in level-sensitive modes, or once per 256 edges in edge-sensitive mode. This allows you to take longer-running measurements, although the resolution is still 16 bits.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pwm/measure_duty_cycle/measure_duty_cycle.c Lines 19 - 37

```
19 float measure_duty_cycle(uint gpio) {
20     // Only the PWM B pins can be used as inputs.
21     assert(pwm_gpio_to_channel(gpio) == PWM_CHAN_B);
22     uint slice_num = pwm_gpio_to_slice_num(gpio);
23
24     // Count once for every 100 cycles the PWM B input is high
25     pwm_config cfg = pwm_get_default_config();
26     pwm_config_set_clkdiv_mode(&cfg, PWM_DIV_B_HIGH);
27     pwm_config_set_clkdiv(&cfg, 100);
28     pwm_init(slice_num, &cfg, false);
29     gpio_set_function(gpio, GPIO_FUNC_PWM);
30
31     pwm_set_enabled(slice_num, true);
32     sleep_ms(10);
33     pwm_set_enabled(slice_num, false);
34     float counting_rate = clock_get_hz(clk_sys) / 100;
35     float max_possible_count = counting_rate * 0.01;
36     return pwm_get_counter(slice_num) / max_possible_count;
37 }
```

12.5.2.6. Configuring PWM period

When free-running, use the following three parameters to control the period of a PWM slice's output (measured in system clock cycles):

- The `TOP` register, which controls the maximum value of the counting period
- The `CSR_PH_CORRECT` bit, which enables phase-correct mode
- The `DIV` register, which controls the clock divider

The slice counts from 0 to `TOP`, then either wraps or begins counting backward, depending on the setting of `CSR_PH_CORRECT`. The clock divider slows the rate of counting, with a maximum speed of one count per cycle, and a minimum speed of one count per 256 cycles. Calculate the period in clock cycles with the following equation:

$$\text{period} = (\text{TOP} + 1) \times (\text{CSR_PH_CORRECT} + 1) \times \left(\text{DIV_INT} + \frac{\text{DIV_FRAC}}{16}\right)$$

To determine the output frequency based on the system clock frequency, use the following equation:

$$f_{PWM} = \frac{f_{sys}}{\text{period}} = \frac{f_{sys}}{(\text{TOP} + 1) \times (\text{CSR_PH_CORRECT} + 1) \times \left(\text{DIV_INT} + \frac{\text{DIV_FRAC}}{16}\right)}$$

Set `DIV_INT` to 0 to divide the count rate by the maximum possible value of 256. You must not set any `DIV_FRAC` bits when `DIV_INT` is 0.

12.5.2.7. Interrupt Request (IRQ) and DMA Data Request (DREQ)

The PWM block has two IRQ outputs. The interrupt status registers `INTR`, `INTS0`, `INTS1`, `INTE0` and `INTE1` allow software to:

- Control which slices assert each of the two IRQs
- Check which slices caused the assertion of an IRQ
- Clear and acknowledge the interrupt

A slice generates an interrupt request each time its counter wraps (or, in phase-correct mode, each time the counter returns to 0). This sets the flag corresponding to this slice in the raw interrupt status register, `INTR`. If this slice's interrupt is enabled in `INTE`, this flag causes the PWM block's IRQ to be asserted, and the flag also appears in the masked interrupt status register `INTS`.

To clear flags, write a mask back to `INTR`. This is demonstrated in the LED fade SDK example below:

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/pwm/led_fade/pwm_led_fade.c

```
/**
 * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
 *
 * SPDX-License-Identifier: BSD-3-Clause
 */

// Fade an LED between low and high brightness. An interrupt handler updates
// the PWM slice's output level each time the counter wraps.

#include "pico/stdlib.h"
#include <stdio.h>
#include "pico/time.h"
#include "hardware/irq.h"
#include "hardware/pwm.h"

void on_pwm_wrap() {
    static int fade = 0;
    static bool going_up = true;
    // Clear the interrupt flag that brought us here
    pwm_clear_irq(pwm_gpio_to_slice_num(PICO_DEFAULT_LED_PIN));

    if (going_up) {
        ++fade;
        if (fade > 255) {
            fade = 255;
            going_up = false;
        }
    } else {
        --fade;
        if (fade < 0) {
            fade = 0;
            going_up = true;
```

```
        }
    }
    // Square the fade value to make the LED's brightness appear more linear
    // Note this range matches with the wrap value
    pwm_set_gpio_level(PICO_DEFAULT_LED_PIN, fade * fade);
}

int main() {
#ifndef PICO_DEFAULT_LED_PIN
#warning pwm/led_fade example requires a board with a regular LED
#else
    // Tell the LED pin that the PWM is in charge of its value.
    gpio_set_function(PICO_DEFAULT_LED_PIN, GPIO_FUNC_PWM);
    // Figure out which slice we just connected to the LED pin
    uint slice_num = pwm_gpio_to_slice_num(PICO_DEFAULT_LED_PIN);

    // Mask our slice's IRQ output into the PWM block's single interrupt line,
    // and register our interrupt handler
    pwm_clear_irq(slice_num);
    pwm_set_irq_enabled(slice_num, true);
    irq_set_exclusive_handler(PWM_DEFAULT_IRQ_NUM(), on_pwm_wrap);
    irq_set_enabled(PWM_DEFAULT_IRQ_NUM(), true);

    // Get some sensible defaults for the slice configuration. By default, the
    // counter is allowed to wrap over its maximum range (0 to 2**16-1)
    pwm_config config = pwm_get_default_config();
    // Set divider, reduces counter clock to sysclock/this value
    pwm_config_set_clkdiv(&config, 4.f);
    // Load the configuration into our PWM slice, and set it running.
    pwm_init(slice_num, &config, true);

    // Everything after this point happens in the PWM interrupt handler, so we
    // can twiddle our thumbs
    while (1)
        tight_loop_contents();
#endif
}
```

This scheme allows multiple slices to generate interrupts concurrently. A system interrupt handler determines which slices caused the most recent interruption, and handles them appropriately. Normally, this means reloading those slices' `TOP` or `CC` registers, but the PWM block can also be used as a source of regular interrupt requests for non-PWM purposes.

The same pulse which sets the interrupt flag in `INTR` is also available as a one-cycle data request to the RP2350 system DMA. For each cycle the DMA sees a DREQ asserted, it makes one data transfer to its programmed location in as timely a manner as possible. Combined with the double-buffered behaviour of `CC` and `TOP`, the DMA can efficiently stream data to a PWM slice at a rate of one transfer per counter period. Alternatively, a PWM slice could serve as a pacing timer for DMA transfers to some other memory-mapped hardware.

12.5.2.8. On-the-fly phase adjustment

For some applications, it is necessary to control the phase relationship between two PWM outputs on different slices.

The global enable register `EN` contains an alias of the `CSR_EN` flag for each slice. Use this register to start and stop several slices simultaneously. If two slices with the same output frequency start at the same time, they run in perfect lockstep, with a fixed phase relationship determined by the initial counter values.

The `CSR_PH_ADV` and `CSR_PH_RET` fields advance or retard a slice's output phase by one count whilst it is running. They do so by inserting or deleting pulses from the clock enable (the output of the clock divider), as shown in Figure 121.

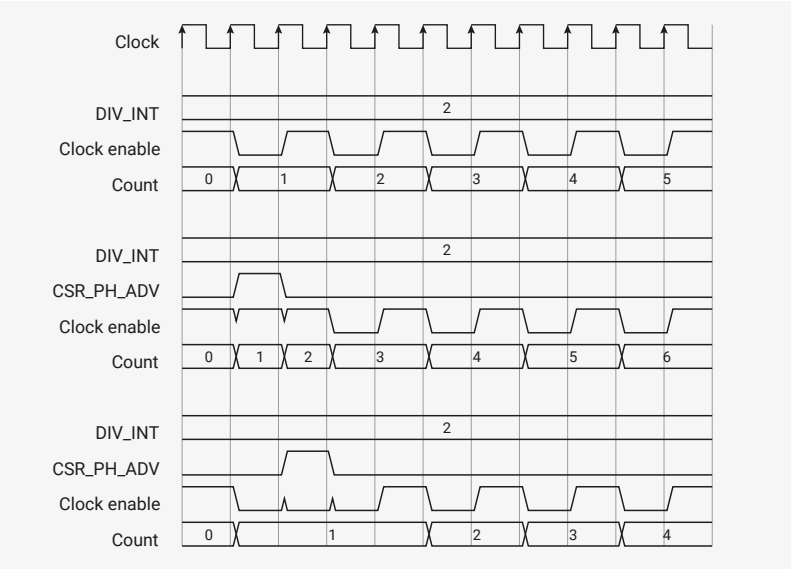

Figure 121. The clock enable signal, output by the clock divider, controls the rate of counting. Phase advance forces the clock enable high on cycles where it is low, causing the counter to jump forward by one count. Phase retard forces the clock enable low when it would be high, holding the counter back by one count.

The counter cannot count faster than once per cycle, so `PH_ADV` requires `DIV_INT` > 1 or `DIV_FRAC` > 0. Likewise, the counter will not start to count backward if `PH_RET` is asserted when the clock enable is permanently low.

To advance or retard the phase by one count, software writes 1 to `PH_ADV` or `PH_RET`. Once an enable pulse has been inserted or deleted, the `PH_ADV` or `PH_RET` register bit returns to 0. Software can poll `CSR` until this happens. `PH_ADV` always inserts a pulse into the next available gap; `PH_RET` always deletes the next available pulse.

12.5.3. List of registers

The PWM registers start at a base address of `0x400a8000` (defined as PWM_BASE in the SDK).

Table 1131. List of PWM registers

Offset	Name	Info
0x000	CH0_CSR	Control and status register
0x004	CH0_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x008	CH0_CTR	Direct access to the PWM counter
0x00c	CH0_CC	Counter compare values
0x010	CH0_TOP	Counter wrap value
0x014	CH1_CSR	Control and status register
0x018	CH1_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x01c	CH1_CTR	Direct access to the PWM counter
0x020	CH1_CC	Counter compare values
0x024	CH1_TOP	Counter wrap value
0x028	CH2_CSR	Control and status register
0x02c	CH2_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.

Offset	Name	Info
0x030	CH2_CTR	Direct access to the PWM counter
0x034	CH2_CC	Counter compare values
0x038	CH2_TOP	Counter wrap value
0x03c	CH3_CSR	Control and status register
0x040	CH3_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x044	CH3_CTR	Direct access to the PWM counter
0x048	CH3_CC	Counter compare values
0x04c	CH3_TOP	Counter wrap value
0x050	CH4_CSR	Control and status register
0x054	CH4_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x058	CH4_CTR	Direct access to the PWM counter
0x05c	CH4_CC	Counter compare values
0x060	CH4_TOP	Counter wrap value
0x064	CH5_CSR	Control and status register
0x068	CH5_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x06c	CH5_CTR	Direct access to the PWM counter
0x070	CH5_CC	Counter compare values
0x074	CH5_TOP	Counter wrap value
0x078	CH6_CSR	Control and status register
0x07c	CH6_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x080	CH6_CTR	Direct access to the PWM counter
0x084	CH6_CC	Counter compare values
0x088	CH6_TOP	Counter wrap value
0x08c	CH7_CSR	Control and status register
0x090	CH7_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x094	CH7_CTR	Direct access to the PWM counter
0x098	CH7_CC	Counter compare values
0x09c	CH7_TOP	Counter wrap value
0x0a0	CH8_CSR	Control and status register

Offset	Name	Info
0x0a4	CH8_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x0a8	CH8_CTR	Direct access to the PWM counter
0x0ac	CH8_CC	Counter compare values
0x0b0	CH8_TOP	Counter wrap value
0x0b4	CH9_CSR	Control and status register
0x0b8	CH9_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x0bc	CH9_CTR	Direct access to the PWM counter
0x0c0	CH9_CC	Counter compare values
0x0c4	CH9_TOP	Counter wrap value
0x0c8	CH10_CSR	Control and status register
0x0cc	CH10_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x0d0	CH10_CTR	Direct access to the PWM counter
0x0d4	CH10_CC	Counter compare values
0x0d8	CH10_TOP	Counter wrap value
0x0dc	CH11_CSR	Control and status register
0x0e0	CH11_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x0e4	CH11_CTR	Direct access to the PWM counter
0x0e8	CH11_CC	Counter compare values
0x0ec	CH11_TOP	Counter wrap value
0x0f0	EN	This register aliases the CSR_EN bits for all channels. Writing to this register allows multiple channels to be enabled or disabled simultaneously, so they can run in perfect sync. For each channel, there is only one physical EN register bit, which can be accessed through here or CHx_CSR.
0x0f4	INTR	Raw Interrupts
0x0f8	IRQ0_INTE	Interrupt Enable for irq0
0x0fc	IRQ0_INTF	Interrupt Force for irq0
0x100	IRQ0_INTS	Interrupt status after masking & forcing for irq0
0x104	IRQ1_INTE	Interrupt Enable for irq1
0x108	IRQ1_INTF	Interrupt Force for irq1
0x10c	IRQ1_INTS	Interrupt status after masking & forcing for irq1

PWM: CH0_CSR, CH1_CSR, …, CH10_CSR, CH11_CSR Registers

Offsets: 0x000, 0x014, …, 0x0c8, 0x0dc

Description

Control and status register

Table 1132. CH0_CSR, CH1_CSR, …, CH10_CSR, CH11_CSR Registers

Bits	Description	Type	Reset
31:8	Reserved.	-	-
7	**PH_ADV**: Advance the phase of the counter by 1 count, while it is running. Self-clearing. Write a 1, and poll until low. Counter must be running at less than full speed (div_int + div_frac / 16 > 1)	SC	0x0
6	**PH_RET**: Retard the phase of the counter by 1 count, while it is running. Self-clearing. Write a 1, and poll until low. Counter must be running.	SC	0x0
5:4	**DIVMODE**	RW	0x0
	Enumerated values:		
	0x0 → DIV: Free-running counting at rate dictated by fractional divider		
	0x1 → LEVEL: Fractional divider operation is gated by the PWM B pin.		
	0x2 → RISE: Counter advances with each rising edge of the PWM B pin.		
	0x3 → FALL: Counter advances with each falling edge of the PWM B pin.		
3	**B_INV**: Invert output B	RW	0x0
2	**A_INV**: Invert output A	RW	0x0
1	**PH_CORRECT**: 1: Enable phase-correct modulation. 0: Trailing-edge	RW	0x0
0	**EN**: Enable the PWM channel.	RW	0x0

PWM: CH0_DIV, CH1_DIV, …, CH10_DIV, CH11_DIV Registers

Offsets: 0x004, 0x018, …, 0x0cc, 0x0e0

Description

INT and FRAC form a fixed-point fractional number.
Counting rate is system clock frequency divided by this number.
Fractional division uses simple 1st-order sigma-delta.

Table 1133. CH0_DIV, CH1_DIV, …, CH10_DIV, CH11_DIV Registers

Bits	Description	Type	Reset
31:12	Reserved.	-	-
11:4	**INT**	RW	0x01
3:0	**FRAC**	RW	0x0

PWM: CH0_CTR, CH1_CTR, …, CH10_CTR, CH11_CTR Registers

Offsets: 0x008, 0x01c, …, 0x0d0, 0x0e4

Table 1134. CH0_CTR, CH1_CTR, …, CH10_CTR, CH11_CTR Registers

Bits	Description	Type	Reset
31:16	Reserved.	-	-
15:0	Direct access to the PWM counter	RW	0x0000

PWM: CH0_CC, CH1_CC, …, CH10_CC, CH11_CC Registers

Offsets: 0x00c, 0x020, …, 0x0d4, 0x0e8

Description

Counter compare values

Table 1135. CH0_CC, CH1_CC, …, CH10_CC, CH11_CC Registers

Bits	Description	Type	Reset
31:16	**B**	RW	0x0000
15:0	**A**	RW	0x0000

PWM: CH0_TOP, CH1_TOP, …, CH10_TOP, CH11_TOP Registers

Offsets: 0x010, 0x024, …, 0x0d8, 0x0ec

Table 1136. CH0_TOP, CH1_TOP, …, CH10_TOP, CH11_TOP Registers

Bits	Description	Type	Reset
31:16	Reserved.	-	-
15:0	Counter wrap value	RW	0xffff

PWM: EN Register

Offset: 0x0f0

Description

This register aliases the CSR_EN bits for all channels.
Writing to this register allows multiple channels to be enabled or disabled simultaneously, so they can run in perfect sync.
For each channel, there is only one physical EN register bit, which can be accessed through here or CHx_CSR.

Table 1137. EN Register

Bits	Description	Type	Reset
31:12	Reserved.	-	-
11	**CH11**	RW	0x0
10	**CH10**	RW	0x0
9	**CH9**	RW	0x0
8	**CH8**	RW	0x0
7	**CH7**	RW	0x0
6	**CH6**	RW	0x0
5	**CH5**	RW	0x0
4	**CH4**	RW	0x0
3	**CH3**	RW	0x0
2	**CH2**	RW	0x0
1	**CH1**	RW	0x0

Bits	Description	Type	Reset
0	**CH0**	RW	0x0

PWM: INTR Register

Offset: 0x0f4

Description

Raw Interrupts

Table 1138. INTR Register

Bits	Description	Type	Reset
31:12	Reserved.	-	-
11	**CH11**	WC	0x0
10	**CH10**	WC	0x0
9	**CH9**	WC	0x0
8	**CH8**	WC	0x0
7	**CH7**	WC	0x0
6	**CH6**	WC	0x0
5	**CH5**	WC	0x0
4	**CH4**	WC	0x0
3	**CH3**	WC	0x0
2	**CH2**	WC	0x0
1	**CH1**	WC	0x0
0	**CH0**	WC	0x0

PWM: IRQ0_INTE Register

Offset: 0x0f8

Description

Interrupt Enable for irq0

Table 1139. IRQ0_INTE Register

Bits	Description	Type	Reset
31:12	Reserved.	-	-
11	**CH11**	RW	0x0
10	**CH10**	RW	0x0
9	**CH9**	RW	0x0
8	**CH8**	RW	0x0
7	**CH7**	RW	0x0
6	**CH6**	RW	0x0
5	**CH5**	RW	0x0
4	**CH4**	RW	0x0
3	**CH3**	RW	0x0
2	**CH2**	RW	0x0

Bits	Description	Type	Reset
1	**CH1**	RW	0x0
0	**CH0**	RW	0x0

PWM: IRQ0_INTF Register

Offset: 0x0fc

Description

Interrupt Force for irq0

Table 1140. IRQ0_INTF Register

Bits	Description	Type	Reset
31:12	Reserved.	-	-
11	**CH11**	RW	0x0
10	**CH10**	RW	0x0
9	**CH9**	RW	0x0
8	**CH8**	RW	0x0
7	**CH7**	RW	0x0
6	**CH6**	RW	0x0
5	**CH5**	RW	0x0
4	**CH4**	RW	0x0
3	**CH3**	RW	0x0
2	**CH2**	RW	0x0
1	**CH1**	RW	0x0
0	**CH0**	RW	0x0

PWM: IRQ0_INTS Register

Offset: 0x100

Description

Interrupt status after masking & forcing for irq0

Table 1141. IRQ0_INTS Register

Bits	Description	Type	Reset
31:12	Reserved.	-	-
11	**CH11**	RO	0x0
10	**CH10**	RO	0x0
9	**CH9**	RO	0x0
8	**CH8**	RO	0x0
7	**CH7**	RO	0x0
6	**CH6**	RO	0x0
5	**CH5**	RO	0x0
4	**CH4**	RO	0x0
3	**CH3**	RO	0x0

Bits	Description	Type	Reset
2	**CH2**	RO	0x0
1	**CH1**	RO	0x0
0	**CH0**	RO	0x0

PWM: IRQ1_INTE Register

Offset: 0x104

Description

Interrupt Enable for irq1

Table 1142. IRQ1_INTE Register

Bits	Description	Type	Reset
31:12	Reserved.	-	-
11	**CH11**	RW	0x0
10	**CH10**	RW	0x0
9	**CH9**	RW	0x0
8	**CH8**	RW	0x0
7	**CH7**	RW	0x0
6	**CH6**	RW	0x0
5	**CH5**	RW	0x0
4	**CH4**	RW	0x0
3	**CH3**	RW	0x0
2	**CH2**	RW	0x0
1	**CH1**	RW	0x0
0	**CH0**	RW	0x0

PWM: IRQ1_INTF Register

Offset: 0x108

Description

Interrupt Force for irq1

Table 1143. IRQ1_INTF Register

Bits	Description	Type	Reset
31:12	Reserved.	-	-
11	**CH11**	RW	0x0
10	**CH10**	RW	0x0
9	**CH9**	RW	0x0
8	**CH8**	RW	0x0
7	**CH7**	RW	0x0
6	**CH6**	RW	0x0
5	**CH5**	RW	0x0
4	**CH4**	RW	0x0